

Grado en Ciencia e Ingeniería de Datos

Bases de Datos: integración y arquitecturas 2026-27

Práctica de laboratorio 1: Generación de datos sintéticos y almacenamiento de información

Entrega: semana 5 de octubre de 2026, el día antes de la clase de prácticas (ver Moodle)

Modelo de trabajo: la práctica se abordará y entregará por parejas

Objetivos de aprendizaje:

- Aprender a generar datos sintéticos para realizar una ingesta de datos coherente con las necesidades reales del sistema a medir.
- Aprender a almacenar la información en diversos tipos de ficheros y Sistemas Gestores de Bases de Datos (SGBD).
- Aprender a automatizar el proceso de generación de datos sintéticos de forma que el código pueda reutilizarse con diferentes cantidades y formatos de datos.
- Aprender a manejarse de forma autónoma en la lectura y comprensión de la documentación técnica de sistemas gestores de bases de datos, librerías y módulos de programación para el diseño e implementación de software que las emplee.

Descripción:

En el marco de esta práctica, los estudiantes deberán desarrollar e implementar un código que permita la generación de datos sintéticos, así como su almacenamiento en diferentes formatos de ficheros y Sistemas Gestores de Bases de Datos (SGBD).

Al finalizar la actividad, se deberá realizar la entrega del código desarrollado junto con un informe técnico que detalle las decisiones tomadas durante el proceso de implementación y las conclusiones obtenidas.

Estas prácticas permitirán a los estudiantes abordar temas clave en bases de datos, como la generación de datos y la relación entre los mismos.

En esta práctica se deberán generar conjuntos de datos relacionales y almacenar la información en diversos tipos de ficheros y bases de datos.

En particular, los estudiantes deben implementar el código que permita:

1. Generar dos conjuntos de datos empleando el paquete Faker¹, donde uno de ellos tenga datos vinculados con el otro:
 - 1.1. El primero, de tamaño parametrizable, contendrá datos de usuarios (nombre, DNI, email, número de teléfono móvil, número de teléfono fijo, dirección, ciudad, código postal² y provincia³). Los datos de ciudad y código postal deberán ser coherentes entre sí empleando los datasets indicados a pie de página. La provincia deberá obtenerse de forma consistente con el código postal. La dirección (calle, número, bloque, etc.) podrá ser generada arbitrariamente con Faker.
 - 1.2. El segundo, deberá generarse a partir del conjunto de usuarios, y contendrá datos de vehículos que hayan pertenecido o pertenezcan a usuarios, empleando el DNI como identificador del usuario. De cada vehículo se generará: matrícula, número de bastidor –secuencia de 17 letras y números–, año, fabricante, modelo, categoría –urbanos, sedán, berlina, cupé, descapotable, deportivo, todoterreno, monovolumen, SUV, furgoneta–.
 - 1.2.1. Para cada usuario se generará un número de vehículos siguiendo una distribución de Poisson con $\lambda=0.9$ truncada a $[0, 4]$.
 - 1.2.2. Deberá implementarse al menos un Provider propio de Faker para generar información no disponible directamente en la biblioteca.
 - 1.2.3. Recuerda que las matrículas de vehículos fabricados hasta 1999 deberán seguir el formato histórico español (por ejemplo, M-1234-AB) y que las de vehículos fabricados a partir del año 2000 deberán seguir el formato nacional actual (por ejemplo, 1234-BCD).
 - 1.2.4. El fabricante, modelo, categoría y periodo de producción deberán obtenerse a partir del fichero JSON de vehículos suministrado junto a la práctica.
2. Almacenar la información generada en ficheros:
 - 2.1. CSV y Parquet⁴: para ambos formatos, se generará un fichero por cada conjunto de datos (usuarios y vehículos) conectando ambos conjuntos de datos por medio de referencias (el campo de la variable de un conjunto de datos será el identificador del otro conjunto de datos).
 - 2.2. JSON⁵ (o JSONL justificando su uso) y Avro⁶: se deberá abordar la tarea de dos modos diferentes:
 - 2.2.1. Un único fichero para todo el conjunto de datos anidando los datos relacionados de forma jerárquica.
 - 2.2.2. Un fichero por cada conjunto de datos (usuarios y vehículos) conectando ambos conjuntos de datos por medio de referencias (el campo de la variable de un conjunto de datos será el identificador del otro conjunto de datos).
 - 2.3. Señala las características más destacadas de cada formato de fichero. ¿Usarías cualquier formato de forma indistinta en cualquier situación? ¿En qué situaciones resultan más aconsejables cada uno de ellos? ¿Qué diferencias encuentras entre usar formatos de almacenamiento basados en filas frente a los basados en columnas? ¿Qué ventajas e inconvenientes encuentras entre almacenar la información mediante ficheros separados o en estructura jerárquica?

¹ <https://faker.readthedocs.io/en/master/index.html>

² https://raw.githubusercontent.com/inigoflores/ds-codigos-postales-ine-es/master/data/codigos_postales_municipios.csv

³ https://es.wikipedia.org/wiki/Anexo:Provincias_de_Espa%C3%B1a_por_c%C3%B3digo_postal

⁴ <https://parquet.apache.org/>

⁵ <https://www.json.org/json-es.html>

⁶ <https://avro.apache.org/>

3. Almacenar la información generada en SGBD:
 - 3.1. SQLite3⁷: sistema de gestión de bases de datos relacional autónomo, sin servidor ni necesidad de configuración, que lee y escribe directamente en archivos de disco.
 - 3.2. PostgreSQL⁸: sistema de gestión de bases de datos relacional. Al contrario que el anterior, requiere de un proceso servidor independiente al que las aplicaciones cliente deberán conectarse.
 - 3.3. MongoDB⁹: sistema de gestión de bases de datos de documentos, que emplea colecciones en lugar de guardar los datos en tablas. Al igual que el anterior, requiere de un proceso servidor independiente al que las aplicaciones cliente deberán conectarse.
 - 3.4. Señala las características más destacadas de cada SGBD. ¿Usarías cualquiera de ellos de forma indistinta en cualquier situación? ¿Para qué situaciones usarías cada uno de ellos?

Consideraciones adicionales:

- **Debe probarse el código** con tamaños de datos para 1.000, 10.000 y 100.000 registros (usuarios).
- **Debe evitarse el uso de Pandas en todos los apartados**, indicando qué implicaciones y/o limitaciones tiene su uso en las diferentes tareas.
- **Debe evitarse abordar la práctica mediante un Notebook de Jupyter con todo el código**. En su lugar, la aplicación deberá **ejecutarse desde línea de comandos** permitiendo especificar, al menos, el número de usuarios a generar, el formato del fichero y directorio de salida de los datos generados, el SGBD a emplear, así como sus parámetros de conexión. Los parámetros podrán proporcionarse mediante argumentos de línea de comandos, fichero de configuración o una combinación de ambos mecanismos.
- **La calidad y claridad del código es parte de la evaluación**. Deberá diseñarse de forma modular y extensible, así como estar comentado para facilitar su comprensión cuando proceda. Se recomienda valorar el uso de patrones de diseño que faciliten la incorporación de nuevos formatos de almacenamiento o nuevos sistemas gestores de bases de datos sin modificar el núcleo de la aplicación. Entre otros, puede resultar de utilidad el **patrón Strategy**¹⁰ para encapsular distintos mecanismos de persistencia, y el **patrón Factory Method**¹¹ para instanciar estrategias seleccionando dinámicamente la implementación adecuada.
- **Deberá elaborarse un informe** con la explicación de lo realizado, las respuestas a las preguntas y reflexiones planteadas, las comparaciones solicitadas, etc. indicando la diferencia entre los diferentes modos de almacenamiento de datos y cómo se pueden utilizar en diferentes escenarios.

Entregable

El entregable de las tareas de laboratorio consiste en un fichero comprimido (**zip** o **tgz**) con la siguiente nomenclatura **BDIA2627-lab1-Nombre1Apellido1_Nombre2Apellido2.zip** (indicando en nombre y apellido de cada integrante). El fichero deberá contener:

- **Subdirectorío src** que contenga los ficheros con el código Python desarrollado. Cada fichero de código fuente deberá indicar el nombre de los autores.
- **Un fichero PDF** que contenga el informe solicitado.

⁷ <https://docs.python.org/3/library/sqlite3.html>

⁸ <https://pypi.org/project/psycopg2/>

⁹ <https://www.mongodb.com/es>

¹⁰ <https://refactoring.guru/es/design-patterns/strategy>

¹¹ <https://refactoring.guru/es/design-patterns/factory-method>